Informe Final - TPO POO Etapa 2

Propuesta elegida: Agencia de Turismo "Rumbo Sur"

Repositorio GitHub: https://github.com/dolocerrudo16/tpo-poo.git

Integrantes:
- Dolores Cerrudo Amoedo
- Germana Mendoza

1. Distribución de Tareas

El desarrollo de la Etapa 2 se dividió de manera equitativa utilizando repositorios y ramas en GitHub para aislar los desarrollos antes de su integración en la rama principal:

Dolores Cerrudo Amoedo:
- Modelado e implementación de las entidades Pasajero (con ordenamiento natural Comparable) y Cliente (con validación de formato de correo y unicidad de documento).
- Jerarquía de la clase abstracta Empleado y subclase Vendedor.
- Lógica central de la clase Reserva, pagos con recargos por cuotas en Pago y cálculo polimórfico en CalculadoraTarifa con exclusión de menores de 3 años en PromocionGrupo.
- mplementación de ControladorReserva para orquestar la creación de reservas, pagos y reemplazo de pasajeros.
- Implementación del módulo de reportes gerenciales en Reportes.
- Desarrollo de la interfaz gráfica: formulario modal interactivo VistaRegistrarCliente (captura y validación de datos con manejo defensivo de ReglaNegocioException) y el punto de entrada MainGUI (inicialización en el Event Dispatch Thread con SwingUtilities.invokeLater y guardado automático mediante ShutdownHook).

Germana Mendoza:
- Modelado e implementación del catálogo de Tour con su jerarquía polimórfica (TourAventura, TourCultural, TourNaturaleza) y contrato de validación de pasajeros (LSP).
- Gestión operativa de la entidad Salida: ciclo de vida completo (PROGRAMADA, CONFIRMADA, COMPLETA, EN_CURSO, FINALIZADA), asignación de recursos (Guia, Vehiculo) y control temporal de inicio.
- Diseño de las políticas polimórficas de cancelación (CancelacionFlexible, CancelacionEstricta, CancelacionPorAgencia) y emisión de NotaDeAjuste.
- Implementación del paquete de excepciones de negocio (ReglaNegocioException, CupoExcedidoException).
- Implementación de ControladorOperaciones y ControladorAdministracion (liquidación de honorarios a guías y persistencia en la agencia).
- Persistencia binaria de objetos en GestorPersistencia (ObjectOutputStream/ObjectInputStream) y diseño del plan de pruebas en PruebaSistema.

- Desarrollo de la interfaz gráfica: ventana principal VistaPrincipal (navegación y disparador de eventos con Swing) y diálogo interactivo VistaProgramarSalida.

2. Cambios respecto del diseño original

Durante la transición del diseño inicial a la codificación final en Java, se realizaron refactorizaciones clave para garantizar el cumplimiento de la arquitectura Modelo-Vista-Controlador (MVC), los principios SOLID y patrones GRASP:

Arquitectura MVC y Controladores (GRASP - Controlador):
Para evitar que la clase central Agencia concentrara un acoplamiento excesivo ("Objeto Dios"), se crearon los controladores ControladorReserva, ControladorOperaciones y ControladorAdministracion. Estos median entre la capa de presentación y el modelo de dominio.

Incorporación de la Capa de Vista (Java Swing):
Se desarrolló una interfaz gráfica de usuario (presentacion.vistas) con manejo de eventos, cumpliendo el requisito técnico de interacción dinámica. Las vistas (VistaPrincipal, VistaProgramarSalida, VistaRegistrarCliente) no contienen lógica de negocio; capturan eventos de usuario y delegan exclusivamente en los controladores.

Delegación de Cálculos (SOLID - SRP) y Experto en Información (GRASP):
La lógica de liquidación tarifaria se desacopló en la clase CalculadoraTarifa. A su vez, Reserva asume el rol de Experto en Información coordinando internamente su propio recálculo total mediante recalcularTotal(), eliminando comportamientos de modelo anémico.

Polimorfismo en Cancelaciones y Promociones (GRASP / SOLID - OCP):
Las devoluciones se estructuraron mediante la interfaz PoliticaCancelacion, y los descuentos comerciales mediante la interfaz Promocion (PromocionAnticipada y PromocionGrupo). Esto permite extender nuevas reglas de negocio sin modificar el código que las invoca.

Transiciones de Estado y Control Temporal en Salida:
Se implementó la mutación automática de Salida a CONFIRMADA cuando la cantidad de pasajeros inscriptos alcanza el cupo mínimo del tour, y se restringió el método iniciar() para impedir el inicio de salidas antes de su fechaInicio real.

Corrección Lógica en Reporte de Cancelaciones (CU11):
Se refactorizó el método Reportes.cancelaciones() para filtrar sobre la fecha real registrada en cada NotaDeAjuste, evitando la inconsistencia de evaluar la fecha original de creación de la reserva.

Encapsulamiento Estricto en Colecciones:
Se implementaron copias defensivas (new ArrayList<>(...)) en métodos de consulta como getSalidas() y getClientes() en Agencia, protegiendo el estado interno ante modificaciones no autorizadas.

Persistencia Binaria Homogénea:
Se implementó Serializable en las entidades del dominio para permitir que GestorPersistencia serialice y recupere el grafo completo de objetos de Agencia en el archivo sistema_agencia.dat sin desfasajes de datos.

3. Dificultades Encontradas

Control de Versiones y Trabajo Colaborativo:
La sincronización de cambios en GitHub presentó desafíos iniciales, particularmente al integrar ramas con dependencias cruzadas (como la relación bidireccional entre Reserva y Salida). Requirió establecer una comunicación activa previa a la creación y aprobación de cada Pull Request.

Alineación Estricta Código-UML y Encapsulamiento:
Fue un desafío purgar el código de atributos auxiliares y asegurar que la interfaz pública de las clases respondiera fielmente al diagrama de clases aprobado, manteniendo el encapsulamiento estricto mediante copias defensivas en las colecciones retornadas.

Gestión del Tiempo y Fechas:
El cálculo exacto de edades, vigencias de tarifas superpuestas y plazos de cancelación exigió manipular correctamente la API nativa java.time (LocalDate y ChronoUnit) para evitar inconsistencias por años bisiestos o diferencias de días en cancelaciones morosas (a 10 y 7 días).

Coordinación de Hilos en la Interfaz Gráfica (Swing):
Garantizar que la ejecución de la interfaz gráfica corriera de forma segura sobre el Event Dispatch Thread (EDT) y enlazar un ShutdownHook para asegurar la persistencia automática de Agencia al cerrar la ventana sin requerir una acción manual del usuario.

4. Reflexión del Grupo

El desarrollo de este TPO evidenció de forma práctica la importancia crítica de definir un modelo de dominio sólido antes de escribir la primera línea de código. La aplicación consciente de los patrones GRASP y los principios SOLID desde la fase de diseño facilitó enormemente la distribución de responsabilidades y la modularización. Mantener la capa de presentación aislada de las reglas de negocio a través del patrón MVC garantizó un sistema limpio, mantenible y escalable. Además, el uso obligatorio de Git y GitHub nos permitió vivenciar un entorno de desarrollo profesional, destacando el valor de la documentación, la trazabilidad de commits y el trabajo en equipo organizado.